Василенко Е.Р. 090303-ПИа-о24

Лабораторная работа №21
Итоговый проект: разработка, тестирование и развёртывание прикладной Python-системы

1. Тема и цель работы

Тема: «Ресторан» — итоговый проект по разработке законченного прикладного Python-приложения с современными подходами к архитектуре, хранению данных, тестированию, контейнеризации и автоматизации разработки. Цель — закрепить и систематизировать знания, полученные за курс, пройдя основные этапы: анализ требований, проектирование, разработка, работа с данными, API, тестирование, контейнеризация, CI/CD и документирование.

2. Постановка задачи

Разработать практически применимую систему автоматизации ресторана (Вариант 5), охватывающую полный цикл заказа: от регистрации клиента и формирования каталога блюд до оформления заказа и доставки. Система должна соответствовать минимальным требованиям методики: объектно-ориентированная модель с разделением на логические слои, реляционная БД с минимум тремя связанными сущностями, REST API, валидация входных данных, централизованная обработка ошибок, авторизация с ролями, автоматизированные тесты (минимум 20) с покрытием не ниже 70 %, Docker, Docker Compose, Git-репозиторий с осмысленной историей и CI-pipeline.

3. Описание предметной области

Ресторан обслуживает клиентов, которые просматривают меню, делают заказ из доступных блюд и ожидают доставку. Управляющий (администратор) ведёт каталог блюд и подтверждает заказы. Основные сущности: Customer (клиент), Dish (блюдо), Order (заказ), OrderItem (позиция заказа), Delivery (доставка). Полный цикл заказа: регистрация/вход → выбор блюд из каталога → создание заказа с проверкой бизнес-правил → подтверждение → создание доставки → отслеживание статуса доставки.

4. Функциональные требования

Требование	Реализация
Не менее 3 сущностей	5 сущностей: Customer, Dish, Order, OrderItem, Delivery
Не менее 5 операций API	регистрация, вход, CRUD блюд, создание/отмена заказа, доставка
Бизнес-правила (не менее 3)	нельзя заказать отсутствующее блюдо; нельзя заказать блюдо не в наличии; сумма заказа ≤ 100 000; максимум 20 позиций; одна доставка на заказ; доставка только подтверждённого заказа
Поиск/фильтрация/сортировка	GET /dishes?sort=asc|desc, GET /dishes/available
Операция с проверкой нескольких сущностей	создание заказа: проверка блюд и расчёт суммы по ценам из БД
Авторизация	JWT, роли USER и ADMIN, защита админ-операций
Документация API	Swagger UI (/docs), OpenAPI (/openapi.json)

5. Нефункциональные требования

Безопасность: пароли хранятся в виде bcrypt-хешей, секреты вынесены в переменные окружения (в Git не хранятся). Наблюдаемость: /health, логирование запросов и бизнес-операций. Качество: статический анализ ruff, покрытие тестами не ниже 70 %. Воспроизводимость: запуск одной командой docker compose up --build. Масштабируемость: слойная архитектура позволяет заменять компоненты независимо.

6. Архитектура приложения

Приложение построено как монолит с разделением ответственности на пять слоёв: API (FastAPI-роутеры), сервисный слой (бизнес-правила), репозитории (доступ к данным), модели (SQLAlchemy) и конфигурация. Такая структура исключает размещение бизнес-логики в HTTP-обработчиках и упрощает тестирование каждого уровня (рисунок 21.1).

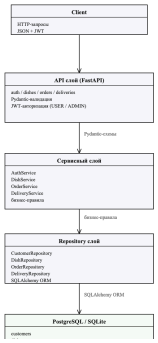


Рисунок 21.1 – Слоистая архитектура приложения «Ресторан»

```
lab_21/
├── src/restaurant/
│   ├── api/            # роутеры: auth, dishes, orders, deliveries, dependencies
│   ├── models/         # SQLAlchemy-модели (5 сущностей)
│   ├── schemas/        # Pydantic-схемы запросов/ответов
│   ├── services/       # бизнес-правила: Auth, Dish, Order, Delivery, security
│   ├── repositories/   # слой доступа к данным
│   ├── exceptions/     # единая модель ошибок
│   ├── config/         # settings, database
│   └── main.py         # сборка приложения, /health, обработка ошибок
├── tests/              # 74 теста (unit, service, mock, API)
├── Dockerfile, docker-compose.yml, .dockerignore
├── .github/workflows/ci.yml
├── requirements.txt, pyproject.toml, .env.example, README.md
```

7. Описание компонентов

API-слой (FastAPI) принимает HTTP-запросы, выполняет валидацию Pydantic и делегирует работу сервисам. Сервисный слой реализует бизнес-правила и координацию репозиториев. Репозитории инкапсулируют SQLAlchemy-запросы, изолируя SQL от сервисов. Модели описывают таблицы БД и связи. Конфигурация (pydantic-settings) читает настройки из переменных окружения.

Файл: src/restaurant/services/services.py — сервисный слой и бизнес-правила

```
"""Сервисный слой: бизнес-правила ресторана."""

import logging
from datetime import date

from sqlalchemy.orm import Session

from restaurant.exceptions.errors import BusinessRuleError, ForbiddenError,
NotFoundError
```

```
from restaurant.models.entities import Customer, Delivery, Dish, Order,
OrderItem
from restaurant.repositories.repositories import (
    CustomerRepository,
    DeliveryRepository,
    DishRepository,
    OrderItemRepository,
    OrderRepository,
)
from restaurant.schemas.schemas import CustomerCreate, DishCreate, DishUpdate,
OrderCreate
from restaurant.services.security import create_access_token, hash_password,
verify_password

logger = logging.getLogger("restaurant")


class AuthService:
    """Регистрация и аутентификация клиентов."""

    def __init__(self, session: Session):
        self.customers = CustomerRepository(session)
        self._session = session

    def register(self, payload: CustomerCreate) -> Customer:
        if self.customers.get_by_email(payload.email) is not None:
            raise BusinessRuleError("EMAIL_ALREADY_REGISTERED", "Пользователь с
таким email уже зарегистрирован")
        customer = Customer(
            email=payload.email,
            name=payload.name,
            hashed_password=hash_password(payload.password),
            role="USER",
        )
        self.customers.add(customer)
        self.customers.commit()
        logger.info("Зарегистрирован клиент %s (%s)", customer.email,
customer.id)
        return customer

    def authenticate(self, email: str, password: str) -> tuple[Customer, str]:
        customer = self.customers.get_by_email(email)
        if customer is None or not verify_password(password,
customer.hashed_password):
            raise ForbiddenError("INVALID_CREDENTIALS", "Неверный email или
пароль")
        token = create_access_token(customer.id, customer.role)
        return customer, token


class DishService:
    """Каталог блюд (CRUD доступен администратору)."""

    def __init__(self, session: Session):
        self.dishes = DishRepository(session)
        self._session = session

    def list(self, sort: str | None = None) -> list[Dish]:
        return self.dishes.list_sorted(sort) if sort in {"asc", "desc"} else
self.dishes.list()
```

Файл: src/restaurant/repositories/repositories.py — репозитории

```
"""Слой доступа к данным (Repository)."""

from typing import Generic, TypeVar

from sqlalchemy import select
from sqlalchemy.orm import Session

from restaurant.models.entities import Customer, Delivery, Dish, Order,
OrderItem
```

```
T = TypeVar("T")


class BaseRepository(Generic[T]):
    model: type[T]

    def __init__(self, session: Session):
        self._session = session

    def get(self, entity_id: int) -> T | None:
        return self._session.get(self.model, entity_id)

    def list(self) -> list[T]:
        return list(self._session.execute(select(self.model)).scalars())

    def add(self, entity: T) -> T:
        self._session.add(entity)
        return entity

    def delete(self, entity: T) -> None:
        self._session.delete(entity)

    def commit(self) -> None:
        self._session.commit()


class CustomerRepository(BaseRepository[Customer]):
    model = Customer

    def get_by_email(self, email: str) -> Customer | None:
        return self._session.execute(select(Customer).where(Customer.email ==
email)).scalar_one_or_none()
```

8. Структура базы данных

Используется реляционная БД PostgreSQL (в Docker) или SQLite (для тестов). Пять таблиц со связями: Customer 1:N Order, Order 1:N OrderItem, Dish 1:N OrderItem, Order 1:1 Delivery (рисунок 21.2). Первичные и внешние ключи, ограничения и уникальность email клиента и order_id доставки заданы на уровне модели.

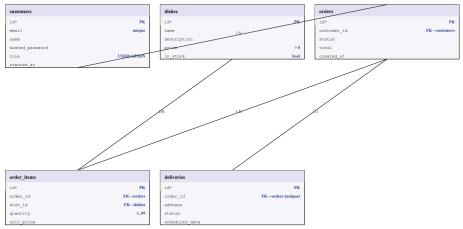


Рисунок 21.2 – Схема базы данных «Ресторан»

Файл: src/restaurant/models/entities.py — модели

```
"""Декларативная ORM-модель базы данных ресторана."""

from datetime import date, datetime

from sqlalchemy import Date, DateTime, Float, ForeignKey, Integer, String, func
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship


class Base(DeclarativeBase):
    pass


class Customer(Base):
    __tablename__ = "customers"

    id: Mapped[int] = mapped_column(primary_key=True)
    email: Mapped[str] = mapped_column(String(255), unique=True, index=True,
nullable=False)
    name: Mapped[str] = mapped_column(String(120), nullable=False)
    hashed_password: Mapped[str] = mapped_column(String(255), nullable=False)
    role: Mapped[str] = mapped_column(String(20), default="USER",
nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now())

    orders: Mapped[list["Order"]] = relationship(back_populates="customer")


class Dish(Base):
    __tablename__ = "dishes"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(120), nullable=False, index=True)
    description: Mapped[str] = mapped_column(String(500), default="",
nullable=False)
    price: Mapped[float] = mapped_column(Float, nullable=False)
    in_stock: Mapped[bool] = mapped_column(default=True, nullable=False)

    order_items: Mapped[list["OrderItem"]] = relationship(back_populates="dish")
```

```
class Order(Base):
    __tablename__ = "orders"

    id: Mapped[int] = mapped_column(primary_key=True)
    customer_id: Mapped[int] = mapped_column(ForeignKey("customers.id"),
nullable=False)
    status: Mapped[str] = mapped_column(String(30), default="CREATED",
nullable=False)
    total: Mapped[float] = mapped_column(Float, default=0, nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now())

    customer: Mapped[Customer] = relationship(back_populates="orders")
    items: Mapped[list["OrderItem"]] = relationship(back_populates="order",
cascade="all, delete-orphan")
    delivery: Mapped["Delivery | None"] = relationship(back_populates="order",
uselist=False)


class OrderItem(Base):
    __tablename__ = "order_items"

    id: Mapped[int] = mapped_column(primary_key=True)
```

9. Описание API

API содержит 13 маршрутов. Публичные: GET /health, POST /auth/register, POST /auth/login, GET /dishes, GET /dishes/available, GET /dishes/{id}. Требуют авторизации: POST/GET /orders, GET /orders/{id}, POST /orders/{id}/cancel, POST /deliveries/orders/{id}, GET /deliveries/orders/{id}, PATCH /deliveries/{id}. Только для ADMIN: POST/PUT/DELETE /dishes, GET /orders/all, POST /orders/{id}/confirm. Документация доступна в Swagger UI.

Метод	Путь	Назначение	Код успеха	Роль
GET	/health	проверка работоспособности	200	все
POST	/auth/register	регистрация клиента	201	все
POST	/auth/login	вход, выдача JWT	200	все
GET	/dishes?sort=asc	каталог с сортировкой	200	все
POST	/dishes	создание блюда	201	ADMIN
PUT	/dishes/{id}	изменение блюда	200	ADMIN
DELETE	/dishes/{id}	удаление блюда	204	ADMIN
POST	/orders	создание заказа	201	USER
GET	/orders	мои заказы	200	USER
POST	/orders/{id}/cancel	отмена заказа	200	USER
POST	/orders/{id}/confirm	подтверждение заказа	200	ADMIN
POST	/deliveries/orders/{id}	создание доставки	201	USER
PATCH	/deliveries/{id}	смена статуса доставки	200	USER

10. Описание бизнес-логики

Бизнес-правила реализованы в сервисном слое: заказ нельзя создать для несуществующего блюда (DISH_NOT_FOUND); нельзя заказать блюдо не в наличии (DISH_OUT_OF_STOCK); сумма заказа не может превышать 100 000 руб. (ORDER_TOO_EXPENSIVE); в заказе максимум 20 позиций (TOO_MANY_ITEMS); доставку можно создать только один раз и только для подтверждённого заказа; отмена недопустима для доставленного или уже отменённого заказа; доступ к чужому заказу запрещён (ORDER_ACCESS_DENIED). Каждое правило покрыто тестами.

Файл: src/restaurant/services/services.py — OrderService

```
        return self.dishes.list_sorted(sort) if sort in {"asc", "desc"} else
self.dishes.list()

    def list_available(self) -> list[Dish]:
        return self.dishes.list_available()

    def get(self, dish_id: int) -> Dish:
        dish = self.dishes.get(dish_id)
        if dish is None:
            raise NotFoundError("DISH_NOT_FOUND", f"Блюдо с id {dish_id} не
найдено")
        return dish

    def create(self, payload: DishCreate) -> Dish:
        dish = Dish(**payload.model_dump())
        self.dishes.add(dish)
        self.dishes.commit()
        logger.info("Создано блюдо %s (id=%s, цена=%.2f)", dish.name, dish.id,
dish.price)
        return dish

    def update(self, dish_id: int, payload: DishUpdate) -> Dish:
        dish = self.get(dish_id)
        for field, value in payload.model_dump(exclude_unset=True).items():
            setattr(dish, field, value)
        self.dishes.commit()
        return dish

    def delete(self, dish_id: int) -> None:
        dish = self.get(dish_id)
        self.dishes.delete(dish)
        self.dishes.commit()


class OrderService:
    """Оформление заказа с бизнес-правилами."""

    def __init__(self, session: Session):
        self.orders = OrderRepository(session)
        self.items = OrderItemRepository(session)
        self.dishes = DishRepository(session)
        self.deliveries = DeliveryRepository(session)
        self._session = session

    def create(self, customer_id: int, payload: OrderCreate) -> Order:
        if len(payload.items) > 20:
            raise BusinessRuleError("TOO_MANY_ITEMS", "Заказ не может содержать
более 20 позиций")

        order = Order(customer_id=customer_id, status="CREATED", total=0)
```

```
        self.orders.add(order)
        total = 0.0
        for item in payload.items:
            dish = self.dishes.get(item.dish_id)
            if dish is None:
                raise NotFoundError("DISH_NOT_FOUND", f"Блюдо с id
{item.dish_id} не найдено")
            if not dish.in_stock:
                raise BusinessRuleError("DISH_OUT_OF_STOCK", f"Блюдо
«{dish.name}» отсутствует в меню")
            unit_price = dish.price
            self.items.add(OrderItem(order=order, dish_id=dish.id,
quantity=item.quantity, unit_price=unit_price))
            total += unit_price * item.quantity
        if total > 100_000:
            raise BusinessRuleError("ORDER_TOO_EXPENSIVE", "Сумма заказа
превышает 100 000 руб.")

        order.total = total
        order.status = "CONFIRMED"
        self.orders.commit()
        logger.info("Создан заказ id=%s клиента %s на сумму %.2f", order.id,
customer_id, total)
        return order

    def get_for_customer(self, order_id: int, customer_id: int, is_admin: bool =
False) -> Order:
        order = self.orders.get(order_id)
        if order is None:
            raise NotFoundError("ORDER_NOT_FOUND", f"Заказ с id {order_id} не
найден")
        if not is_admin and order.customer_id != customer_id:
```

11. Авторизация

Реализованы два механизма: bcrypt-хеширование паролей (никогда не хранятся открыто) и JWT-токены (PyJWT, алгоритм HS256, срок жизни 60 минут). Роли USER и ADMIN имеют разные права: пользователь создаёт и отменяет собственные заказы, администратор управляет каталогом и подтверждает заказы. Проверка токена и роли выполняется зависимостями FastAPI (get_current_customer, require_admin).

Файл: src/restaurant/services/security.py — пароли и JWT

```
"""Идентификация пользователя: пароли, JWT-токены, роли."""

from datetime import UTC, datetime, timedelta

import bcrypt
import jwt

from restaurant.config.settings import get_settings


def hash_password(password: str) -> str:
    return bcrypt.hashpw(password.encode("utf-8"),
bcrypt.gensalt()).decode("utf-8")


def verify_password(plain: str, hashed: str) -> bool:
    try:
        return bcrypt.checkpw(plain.encode("utf-8"), hashed.encode("utf-8"))
    except ValueError:
        return False
```

```
def create_access_token(customer_id: int, role: str) -> str:
    settings = get_settings()
    expire = datetime.now(UTC) +
timedelta(minutes=settings.access_token_expire_minutes)
    payload = {"sub": str(customer_id), "role": role, "exp": expire}
    return jwt.encode(payload, settings.secret_key,
algorithm=settings.jwt_algorithm)


def decode_token(token: str) -> dict:
    settings = get_settings()
    try:
        return jwt.decode(token, settings.secret_key,
algorithms=[settings.jwt_algorithm])
    except jwt.PyJWTError as exc:
        raise ValueError("Некорректный или просроченный токен") from exc


def role_from_token(token: str) -> str:
    return decode_token(token).get("role", "USER")
```

12. Обработка ошибок

Введена единая модель ошибок: классы AppError, NotFoundError (404), BusinessRuleError (409), ForbiddenError (403). Глобальные обработчики FastAPI преобразуют исключения в JSON-ответ без traceback: {"error": "CODE", "message": "..."}. Ошибки валидации Pydantic возвращаются с кодом 422 и списком полей.

Файл: src/restaurant/exceptions/errors.py — модель ошибок

```
"""Исключения приложения и их отображение в HTTP-ответы."""

from dataclasses import dataclass


@dataclass
class AppError(Exception):
    error_code: str
    message: str
    status_code: int = 400


class NotFoundError(AppError):
    def __init__(self, code: str, message: str):
        super().__init__(code, message, 404)


class BusinessRuleError(AppError):
    def __init__(self, code: str, message: str):
        super().__init__(code, message, 409)


class ForbiddenError(AppError):
    def __init__(self, code: str, message: str):
        super().__init__(code, message, 403)
```

Файл: src/restaurant/main.py — обработчики и логирование

```
app.include_router(auth.router)
app.include_router(dishes.router)
app.include_router(orders.router)
app.include_router(deliveries.router)
```

```
@app.exception_handler(AppError)
async def app_error_handler(_: Request, exc: AppError) -> JSONResponse:
    logger.warning("Business error: %s — %s", exc.error_code, exc.message)
    return JSONResponse(status_code=exc.status_code, content={"error":
exc.error_code, "message": exc.message})


@app.exception_handler(RequestValidationError)
async def validation_error_handler(_: Request, exc: RequestValidationError) ->
JSONResponse:
    details = []
    for err in exc.errors():
        loc = ".".join(str(part) for part in err["loc"] if part not in {"body",
"query"})
        details.append({"field": loc or err["loc"][-1], "message": err["msg"]})
    return JSONResponse(status_code=422, content={"error": "VALIDATION_ERROR",
"details": details})


@app.middleware("http")
async def log_requests(request: Request, call_next):
    start = time.perf_counter()
    response = await call_next(request)
    duration_ms = (time.perf_counter() - start) * 1000
    logger.info("%s %s -> %s (%.1f ms)", request.method, request.url.path,
response.status_code, duration_ms)
    return response


@app.get("/health", tags=["system"])
def health() -> dict:
    return {"status": "healthy", "service": "restaurant"}
```

13. Тестирование

Написано 74 автоматических теста: unit-тесты безопасности (хеширование, JWT); тесты сервисного слоя с реальной БД (бизнес-правила, расчёты, параметризованные проверки отсутствующих блюд); тесты с Mock (изоляция сервисов от БД через unittest.mock); интеграционные тесты API (TestClient + SQLite, включая авторизацию, роли, конфликты, валидацию). Тесты выполняются на in-memory SQLite со StaticPool — не требуют Docker.

Файл: tests/test_security.py — unit-тесты

```
"""Unit-тесты безопасности: хеширование паролей, JWT-токены, роли."""

import pytest

from restaurant.services.security import (
    create_access_token,
    decode_token,
    hash_password,
    role_from_token,
    verify_password,
)


class TestPasswordHashing:
    def test_hash_never_plain(self):
        hashed = hash_password("secret123")
```

```
        assert hashed != "secret123"
        assert hashed.startswith("$")

    def test_verify_correct_password(self):
        assert verify_password("secret123", hash_password("secret123")) is True

    def test_verify_wrong_password(self):
        assert verify_password("wrong", hash_password("secret123")) is False

    def test_two_hashes_differ(self):
        assert hash_password("secret123") != hash_password("secret123")


class TestJwt:
    def test_create_and_decode_roundtrip(self):
        token = create_access_token(42, "ADMIN")
        payload = decode_token(token)
        assert payload["sub"] == "42"
        assert payload["role"] == "ADMIN"

    def test_role_from_token(self):
        assert role_from_token(create_access_token(1, "USER")) == "USER"

    def test_decode_invalid_token_raises(self):
```

Файл: tests/test_mocks.py — Mock-тесты

```
"""Тесты с Mock: изоляция сервисного слоя от реальной БД."""

from unittest.mock import MagicMock, patch

import pytest
from pydantic import ValidationError

from restaurant.exceptions.errors import BusinessRuleError, NotFoundError
from restaurant.models.entities import Customer, Dish, Order, OrderItem
from restaurant.schemas.schemas import CustomerCreate, OrderCreate, OrderItemIn
from restaurant.services.services import AuthService, OrderService


class TestAuthServiceWithMock:
    def test_register_mocks_repository(self):
        session = MagicMock()
        service = AuthService(session)
        service.customers = MagicMock()
        service.customers.get_by_email.return_value = None
        service.customers.add.return_value = None

        customer = service.register(CustomerCreate(email="mock@example.com",
name="Мок", password="secret123"))


service.customers.get_by_email.assert_called_once_with("mock@example.com")
        service.customers.add.assert_called_once()
        service.customers.commit.assert_called_once()
        assert customer.role == "USER"

    def test_register_duplicate_with_mock_raises(self):
        session = MagicMock()
        service = AuthService(session)
        service.customers = MagicMock()
        existing = Customer(id=1, email="mock@example.com", name="Существующий",
hashed_password="x")
        service.customers.get_by_email.return_value = existing

        with pytest.raises(BusinessRuleError) as exc:
            service.register(CustomerCreate(email="mock@example.com",
name="Мок", password="secret123"))
        assert exc.value.error_code == "EMAIL_ALREADY_REGISTERED"

    def test_authenticate_with_mock(self):
        session = MagicMock()
```

```
        service = AuthService(session)
        service.customers = MagicMock()
        customer = Customer(
            id=1,
            email="mock@example.com",
            name="Мок",
            hashed_password="$2b$12$abcdefghijklmnopqrstuv",
            role="USER",
        )
        service.customers.get_by_email.return_value = customer

        with (
            patch("restaurant.services.services.verify_password",
return_value=True),
            patch("restaurant.services.services.create_access_token",
return_value="fake-token"),
```

Файл: tests/test_api.py — интеграционные тесты API

```
"""Интеграционные тесты REST API (TestClient + SQLite)."""

import pytest

from tests.conftest import USER_EMAIL, USER_PASSWORD, auth, seed_dishes


class TestHealth:
    def test_health(self, client):
        resp = client.get("/health")
        assert resp.status_code == 200
        assert resp.json()["status"] == "healthy"

    def test_docs_available(self, client):
        assert client.get("/docs").status_code == 200
        assert client.get("/openapi.json").status_code == 200


class TestAuthApi:
    EMAIL = "new@example.com"

    def test_register_success(self, client):
        resp = client.post("/auth/register", json={"email": self.EMAIL, "name":
"Новый", "password": "secret123"})
        assert resp.status_code == 201
        body = resp.json()
        assert body["email"] == self.EMAIL
        assert body["role"] == "USER"

    def test_register_duplicate_conflict(self, client, user_token):
        resp = client.post("/auth/register", json={"email": USER_EMAIL, "name":
"Дубль", "password": "secret123"})
        assert resp.status_code == 409
        assert resp.json()["error"] == "EMAIL_ALREADY_REGISTERED"

    def test_login_success(self, client, user_token):
        resp = client.post("/auth/login", json={"email": USER_EMAIL, "password":
USER_PASSWORD})
        assert resp.status_code == 200
        assert resp.json()["token_type"] == "bearer"
        assert resp.json()["access_token"]

    def test_login_wrong_password_403(self, client):
        resp = client.post("/auth/login", json={"email": USER_EMAIL, "password":
"badpass123"})
        assert resp.status_code == 403
        assert resp.json()["error"] == "INVALID_CREDENTIALS"

    @pytest.mark.parametrize(
        "payload",
        [
            {"email": "bad-email", "name": "Иван", "password": "secret123"},
            {"email": "a@b.ru", "name": "И", "password": "secret123"},
```

```
{"email": "a@b.ru", "name": "Иван", "password": "123"},
```

14. Результаты тестирования

Все 74 теста успешно проходят (74 passed, ~34 с). Команда pytest -v выводит детальный отчёт по каждой проверке (рисунок 21.3).



Рисунок 21.3 – Результаты pytest (74 passed)

15. Анализ покрытия

Покрытие кода тестами — 95 % (покрыто 479 из 502 строк). Не покрыты в основном ветки верхнеуровневых обработчиков и часть редко вызываемых проверок. Требуемый минимум 70 % значительно превышен (рисунок 21.4).

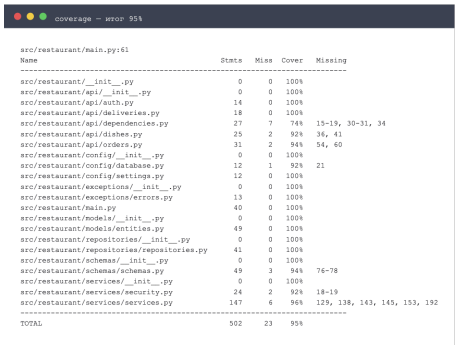


Рисунок 21.4 – Покрытие кода тестами (95 %)

Статический анализ ruff check src tests — без замечаний; код отформатирован ruff format.

16. Docker и Docker Compose

Dockerfile собирает образ на python:3.12-slim: устанавливает зависимости, копирует исходный код, задаёт PYTHONPATH и HEALTHCHECK (/health через urllib, т.к. в slim-образе нет curl). docker-compose.yml описывает два сервиса: app (сборка из Dockerfile, порт 8000, DATABASE_URL через environment, ожидание готовности БД через depends_on condition: service_healthy) и db (postgres:16 с volume postgres_data и healthcheck pg_isready). Данные БД переживают перезапуск контейнеров.

Файл: Dockerfile

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY src ./src

ENV PYTHONPATH=/app/src
ENV PORT=8000
EXPOSE 8000

HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD python -c "import urllib.request; urllib.request.urlopen('http://
localhost:8000/health')"
```

```
CMD ["uvicorn", "restaurant.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Файл: docker-compose.yml

```
services:
  app:
    build: .
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=postgresql+psycopg2://
restaurant:restaurant_password@db:5432/restaurant
      - SECRET_KEY=${SECRET_KEY:-change-me-in-production}
      - ACCESS_TOKEN_EXPIRE_MINUTES=60
    depends_on:
      db:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request;
urllib.request.urlopen('http://localhost:8000/health')"]
      interval: 30s
      timeout: 5s
      retries: 3

  db:
    image: postgres:16
    environment:
      - POSTGRES_DB=restaurant
      - POSTGRES_USER=restaurant
      - POSTGRES_PASSWORD=restaurant_password
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U restaurant -d restaurant"]
      interval: 10s
      timeout: 5s
      retries: 5

volumes:
  postgres_data:
```

Файл: .dockerignore

```
.git
.venv
venv
__pycache__
*.pyc
.pytest_cache
.ruff_cache
.env
.idea
.vscode
*.db
htmlcov
.coverage
```

17. Конфигурация и секреты

Настройки задаются через переменные окружения (pydantic-settings): DATABASE_URL, SECRET_KEY, JWT_ALGORITHM, ACCESS_TOKEN_EXPIRE_MINUTES. Файл .env исключён из Git (.gitignore), в репозитории присутствует шаблон .env.example. Пароль БД задаётся только в docker-compose.yml, не попадая в исходный код.

Файл: src/restaurant/config/settings.py

```
"""Конфигурация приложения через переменные окружения."""

from functools import lru_cache

from pydantic_settings import BaseSettings, SettingsConfigDict


class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env",
env_file_encoding="utf-8", extra="ignore")

    app_name: str = "Restaurant API"
    database_url: str = "sqlite:///./restaurant.db"
    secret_key: str = "change-me-in-production"
    jwt_algorithm: str = "HS256"
    access_token_expire_minutes: int = 60


@lru_cache
def get_settings() -> Settings:
    return Settings()
```

Файл: .env.example

```
DATABASE_URL=postgresql+psycopg2://restaurant:restaurant_password@db:5432/
restaurant
SECRET_KEY=change-me-in-production
JWT_ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60
TEST_DATABASE_URL=sqlite:///./test.db
```

18. CI/CD

Workflow GitHub Actions (.github/workflows/ci.yml) запускается при push в main и pull_request. Этапы: checkout → установка Python 3.12 → установка зависимостей → линтер ruff → запуск тестов с покрытием (pytest --cov, fail-under 70 %) → сборка Docker-образа → проверка корректности docker compose config. Если тесты или линтер не прошли, pipeline завершается ошибкой, и последующие этапы не выполняются.

Файл: .github/workflows/ci.yml

```
name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: |
          pip install --upgrade pip
          pip install -r requirements.txt
```

```
      - name: Lint (ruff)
        run: ruff check src tests

      - name: Run tests with coverage
        run: |
          PYTHONPATH=src pytest tests --cov=src --cov-report=term-missing --cov-
fail-under=70

      - name: Docker build
        run: docker build -t restaurant-app .

      - name: Docker Compose config check
        run: docker compose config
```

19. Демонстрация работы (сквозной сценарий)

Сквозной сценарий выполнен через REST API: (1) GET /health — сервис здоров; (2) регистрация клиента; (3) вход и получение JWT; (4) каталог блюд с сортировкой по цене; (5) создание блюда администратором (201); (6) создание заказа «стейк ×2 + чизкейк» с расчётом суммы 2100 руб.; (7) создание доставки с датой; (8) демонстрация обработки ошибок: 404 для несуществующего блюда, 409 для повторной доставки, 422 для даты в прошлом; (9) просмотр списка заказов (рисунок 21.5).

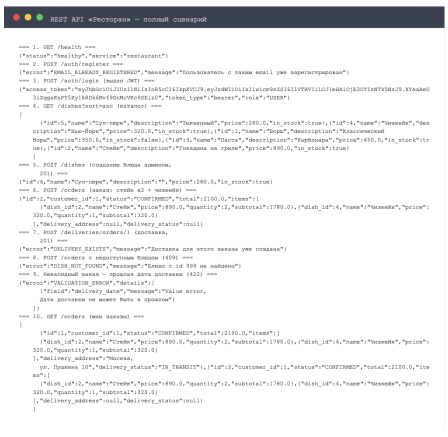


Рисунок 21.5 – REST API: полный сценарий заказа

Журнал запросов приложения (рисунок 21.6) показывает логирование каждого HTTP-запроса с методом, путём, статусом и временем, а также бизнес-

событий (создание блюда, заказа, доставки). Swagger UI доступен по /docs с описанием всех 13 маршрутов.



Рисунок 21.6 – Журнал запросов (logging)

20. Анализ проблем и ограничений

Основные ограничения: механизм событий доставки имитируется синхронными HTTP-вызовами (брокер сообщений не используется — уровень A); JWT без refresh-токенов; отсутствует пагинация каталога при большом количестве блюд. Эти ограничения не влияют на соответствие минимальным требованиям задания.

21. Перспективы развития

Переход на уровень B с добавлением Redis (кэширование каталога и rate limiting); внедрение брокера сообщений для асинхронной обработки событий создания заказа; добавление пагинации, поиска и фильтров; refresh-токены и восстановление пароля; мониторинг метрик (количество запросов, время ответа); расширение тестов эндпоинтов администратора и уведомлений.

22. Вывод

В ходе итогового проекта разработана законченная система «Ресторан»: слойная архитектура (API — сервисы — репозитории — модели), REST API из 13 маршрутов с JWT-авторизацией и двумя ролями, реляционная БД с пятью связанными сущностями, шесть бизнес-правил в сервисном слое, централизованная обработка ошибок, 74 автоматических теста с покрытием 95 %, Docker и Docker Compose, CI-pipeline GitHub Actions и Git-история с осмысленными коммитами. Проект удовлетворяет всем минимальным требованиям методических указаний и готов к демонстрации на защите.